

Engineering Reboot 2026

Day 29 – Expanded Implementation Objectives

Mission

Develop professional Git workflow discipline by inspecting, staging, reviewing, committing, and validating repository changes exactly as a software engineer would in a collaborative development environment.

The emphasis is not learning new Git commands, but learning **when**, **why**, and **how** to use them.

Learning Goals

By the end of today's work you should be able to:

- confidently inspect repository status
 - understand every modified file
 - stage only intended changes
 - compare staged and unstaged work
 - review commit history
 - verify repository integrity before and after commits
 - maintain a clean working tree
 - demonstrate a repeatable Git workflow
-

Exercise 1

Repository Health Inspection

Objective

Determine the complete state of the repository before making any changes.

Skills

- repository inspection
- identifying modified files

- recognizing staged vs unstaged changes

Expected Activities

Run commands that answer questions such as:

- Is my repository clean?
- Which files changed?
- Which files are tracked?
- Which files are untracked?
- Which branch am I on?
- Is my local branch synchronized with the remote?

Validation

You should be able to explain:

"Exactly what Git knows about my repository."

Exercise 2

Working Tree Analysis

Objective

Understand precisely what changed inside modified files.

Skills

Review differences before staging.

Expected Activities

Inspect

- inserted lines
- deleted lines
- modified lines

Determine whether each change is

- intentional

- temporary
- experimental
- production-ready

Validation

You should be able to justify every modification that exists.

Exercise 3

Selective Staging

Objective

Stage only the changes that belong in the next commit.

Skills

Practice engineering judgment rather than blindly staging everything.

Expected Activities

Evaluate each modified file.

Decide

- stage now
- postpone
- discard
- ignore

Engineering Principle

A commit should represent **one logical unit of work**.

Exercise 4

Review the Staging Area

Objective

Verify the exact contents of the next commit.

Skills

Differentiate between

- staged changes
- unstaged changes

Validation

Before committing, answer:

"What exactly will this commit contain?"

Exercise 5

Commit History Inspection

Objective

Read repository history as documentation.

Skills

Review

- commit sequence
- commit messages
- author
- timestamps

Observe how each commit tells the story of project evolution.

Exercise 6

Repository Comparison

Objective

Compare different repository states.

Skills

Understand the differences between

- Working Tree
- Staging Area (Index)
- HEAD
- Previous commits

Recognize where each version of a file exists.

Exercise 7

Engineering Recovery Practice

Objective

Safely recover from common development mistakes.

Possible scenarios include

- accidentally modifying a file
- staging the wrong file
- reverting unwanted changes
- restoring a previous version

The emphasis is understanding recovery without endangering committed work.

Mini-Project

Professional Git Workflow Demonstration

Scenario

Imagine you have completed a feature and are preparing it for production.

Your task is to perform a complete engineering workflow.

Workflow

1. Inspect repository status
 2. Review modified files
 3. Examine differences
 4. Decide what belongs in the commit
 5. Stage appropriate files
 6. Review staged changes
 7. Commit using a meaningful message
 8. Push to GitHub
 9. Fetch remote updates
 10. Verify repository cleanliness
 11. Confirm synchronization with origin
-

Expected Deliverables

By the end of the mini-project you should have evidence demonstrating:

- Repository inspection
 - File comparison
 - Staging
 - Commit creation
 - Repository history
 - Push success
 - Fetch success
 - Clean working tree
-

Engineering Reflection (Notes.md)

Suggested topics:

- Difference between Working Tree and Staging Area
- Why staging exists
- Importance of reviewing changes before committing
- Why small commits improve software quality
- Git workflow that felt most natural
- Commands that became part of your standard development process

Journal.md

Reflect on:

- What habits felt most professional today?
- Which Git command became significantly clearer?
- What mistakes did you intentionally avoid?
- How has your confidence in Git evolved since Week 1?
- How will today's workflow improve your future software projects?

Success Criteria

You should be able to sit down at any Git repository and confidently answer:

- What has changed?
- What is staged?
- What is not staged?
- What will be committed?
- What has already been committed?
- Is my repository synchronized with the remote?
- Is my repository in a clean, releasable state?